

Lab Work 07

Topic	Introduction to AI Copilots, Prompt Engineering Basics
Objective	Get familiar with using an AI Copilot for basic code assistance and practice writing effective prompts.
AI Copilot Usage	Allowed and encouraged

Important Notes

- This is an individual task. All code must be your own original work.
- Ask questions and seek help from the lecturer or teaching assistants if you are stuck.
- Focus on writing clean, well-organized, and well-documented code.

Submission

- Submit updated ***school_simulator.py*** and your prompt history for this session.

Tasks

1. First Interaction & Docstrings:

- a) Open your ***school_simulator.py*** file.
- b) Select a simple method, like ***Person.get_full_name()***. Copy its definition.
- c) **Prompt Practice:** Paste the method definition into your chosen AI assistant and ask: "Generate a Python docstring for this method, explaining what it does." Evaluate the suggestion.
- d) **Prompt Practice (Refactoring Idea):** Copy the ***calculate_grade*** function. Paste it into your AI assistant and ask: "Suggest a way to refactor this Python function to use a dictionary lookup or a more concise structure instead of multiple if/elif/else statements." Analyse the suggestion (don't necessarily implement it yet).

2. Generating Boilerplate Code:

- a) **Prompt Practice:** Ask your AI assistant: "Generate a Python class 'Course' with a constructor that takes ***course_code*** (string) and ***course_name*** (string). It should also initialize an empty list attribute called ***units***. Include a method ***add_unit*** that takes a ***Unit*** object (assume a ***Unit*** class exists) and adds it to the ***units*** list, but only if it's not already present." Copy the generated code into your ***school_simulator.py*** file.

3. Explaining Code:

- a) Copy the code block for the **PersonFactory.create_person** method (including the **try...except** blocks).
- b) **Prompt Practice:** Paste the code into your AI assistant and ask: "Explain what this Python code does, step by step, including its error handling." Review the explanation for accuracy and clarity.

4. Prompt Refinement:

- a) **Prompt Practice (Vague vs. Specific):**
 - i) Vague Prompt: Ask your AI assistant: "Write a Python function to check student prerequisites." Note the likely generic or assumption-filled output.
 - ii) Specific Prompt: Ask: "Write a Python function **check_prerequisites(student, unit)** that takes a **Student** object and a **Unit** object. Assume the **Unit** object has an attribute **prerequisites** which is a list of unit codes (strings). Assume the **Student** object has an attribute **units_completed** which is also a list of unit codes the student passed. The function should return **True** if the student's **units_completed** contains all codes listed in the unit's **prerequisites**, and **False** otherwise. Handle the case where **prerequisites** is empty or doesn't exist (should return **True**)."
 - iii) Compare the quality and usefulness of the generated code from both prompts.

5. Creative Challenge (Optional, if time permits):

- a) Experiment with **role-playing**. Ask the AI: "Act as an expert Python developer reviewing OOP code. Suggest improvements to the following **Unit.add_student** method for clarity, robustness, and efficiency." Paste your current **add_student** method and analyse the suggestions.